Rotary Dryer Algorithm - Complete Engineering Specification for Devin AI

PROJECT OVERVIEW

This specification documents a **Non-Adiabatic Cocurrent Rotary Dryer** algorithm for industrial salt processing. The system uses a complex nested iteration approach with custom psychrometric calculations to solve coupled heat and mass transfer equations.

Physical System Description

- **Process**: Rotary dryer for salt/mineral products
- **Configuration**: Cocurrent flow (air and solids move in same direction)
- **Operation**: Non-adiabatic (20% heat loss to environment)
- **Application**: Final drying of salt products from centrifuge

Key Engineering Challenges

1. **Coupled Equations**: Heat transfer, mass transfer, and psychrometrics are interdependent
2. **Multiple Unknowns**: 4 nested iteration loops with different convergence criteria
3. **Custom Psychrometrics**: Avoid external libraries using Goal Seek residual method
4. **Physical Constraints**: Temperature, moisture, and mass flow limits must be enforced

---

GOAL SEEK IMPLEMENTATION REQUIREMENTS

Why Goal Seek is the ONLY Working Method

Through extensive testing, the following methods **FAILED**:

- **Newton-Raphson**: Derivatives too unstable near psychrometric boundaries
- **Bisection**: Too slow, often trapped in local minima
- **Secant Method**: Diverged due to function discontinuities
- **Brent's Method**: Could not handle the coupled nature of equations
- **Multi-dimensional solvers**: Failed due to constraint violations

Goal Seek succeeded because it:

- Uses **adaptive step sizing** internally
- **Respects Excel's constraint handling**
- **Maintains physical bounds** automatically
- **Handles discontinuities** in Antoine equation

- **Converges reliably** within specified tolerance

Exact Goal Seek Equivalent Algorithm

python

```python
def excel_goal_seek_equivalent(target_cell_function, changing_variable,
                               goal_value=0.0, tolerance=1e-6, max_iterations=100):
    """
    Replicate Excel's Goal Seek algorithm exactly
    This is the ONLY method that works for this system
    """

    # Excel's Goal Seek algorithm (reverse-engineered)
    x = changing_variable  # Current value
    step = abs(x) * 0.1 if x != 0 else 0.1  # Initial step size

    # Calculate initial function value
    f_current = target_cell_function(x)

    # Determine search direction
    x_test = x + step
    f_test = target_cell_function(x_test)

    if abs(f_test - goal_value) < abs(f_current - goal_value):
        direction = 1  # Positive direction is better
    else:
        direction = -1  # Negative direction is better

    iteration = 0
    previous_x = x
    previous_f = f_current

    while iteration < max_iterations:
        iteration += 1

        # Calculate current function value
        f_current = target_cell_function(x)

        # Check convergence
        if abs(f_current - goal_value) < tolerance:
            return True, x, iteration

        # Excel's adaptive step sizing
        if iteration > 1:
            # Use secant-like update when possible
            if abs(f_current - previous_f) > 1e-12:
                slope = (f_current - previous_f) / (x - previous_x)
                step_new = -(f_current - goal_value) / slope

                # Limit step size to prevent wild jumps
                max_step = abs(x) * 0.5 if x != 0 else 1.0
```

```
                if abs(step_new) > max_step:
                    step_new = max_step * (1 if step_new > 0 else -1)

                step = step_new
            else:
                # Reduce step size if no progress
                step *= 0.5

        # Store previous values
        previous_x = x
        previous_f = f_current

        # Update variable
        x += step

        # Ensure we stay within reasonable bounds
        # (This is critical for psychrometric calculations)
        if 'TW' in str(target_cell_function):  # Wet-bulb temperature
            x = max(10.0, min(200.0, x))
        elif 'T1' in str(target_cell_function):  # Outlet air temperature
            x = max(30.0, min(500.0, x))
        elif 'TS1' in str(target_cell_function):  # Outlet solids temperature
            x = max(50.0, min(300.0, x))

        # Adaptive step size reduction
        if iteration % 10 == 0:
            step *= 0.8  # Reduce step size periodically

    return False, x, iteration  # Failed to converge

def goal_seek_with_bounds(target_function, initial_guess, bounds, goal=0.0):
    """
    Goal Seek with explicit bounds (safer for this application)
    """
    x_min, x_max = bounds
    x = max(x_min, min(x_max, initial_guess))  # Ensure initial guess is within bounds

    success, result, iterations = excel_goal_seek_equivalent(
        target_function, x, goal_value=goal
    )

    # Ensure result is within bounds
    if success:
        result = max(x_min, min(x_max, result))

    return success, result, iterations
```

Mandatory Iteration Sequence Implementation

python

```python
def mandatory_iteration_sequence(vars, constants):
    """
    EXACT iteration sequence - DO NOT MODIFY ORDER!
    This order was found through trial and error and is the only one that works
    """

    # STEP 1: MANDATORY - Restore initial values
    vars = restore_standard_values()

    convergence_achieved = False
    outer_iteration = 0
    max_outer_iterations = 30

    while not convergence_achieved and outer_iteration < max_outer_iterations:
        outer_iteration += 1
        print(f"Outer iteration {outer_iteration}: TS1 = {vars.TS1:.2f}°C")

        # INNER LOOP 1: TW4 (FIRST - MANDATORY ORDER)
        def TW4_residual_function(TW4_guess):
            """Residual function for TW4 Goal Seek: N34 = 0"""
            vars.TW4 = TW4_guess

            # Calculate HS4 (TS4 = TW4 assumption)
            TS4 = TW4_guess
            HS4 = (constants.CS + constants.X2) * TS4

            # Energy balance for H4
            H4 = constants.H2 + (constants.L_actual * (HS4 - constants.HS2) *
                                 (1 + constants.alpha)) / constants.G

            # Calculate T4 from enthalpy
            T4 = (H4 - constants.W2 * constants.Hfg) / (constants.CG + constants.Cpv * constant

            # Psychrometric calculation at TW4
            Psat_TW4 = vapor_pressure_antoine(TW4_guess)
            W_sat_TW4 = absolute_humidity(Psat_TW4, 1.0)
            H_psychrometric = air_enthalpy(TW4_guess, W_sat_TW4)

            # Residual (Excel cell N34)
            residual = H_psychrometric - H4
            return residual

        # Execute TW4 Goal Seek
        success1, TW4_converged, iter1 = goal_seek_with_bounds(
            TW4_residual_function,
            vars.TW4_initial,
```

```
        bounds=(30.0, 150.0)
    )

    if not success1:
        raise ConvergenceError(f"TW4 Goal Seek failed at outer iteration {outer_iteration}"

    vars.TW4 = TW4_converged

    # INNER LOOP 2: TW5 (SECOND - MANDATORY ORDER)
    def TW5_residual_function(TW5_guess):
        """Residual function for TW5 Goal Seek: N39 = 0"""
        vars.TW5 = TW5_guess

        # Use converged TW4 values
        TS4 = vars.TW4
        HS4 = (constants.CS + constants.X2) * TS4

        # Calculate HS5 (TS5 = TW5 assumption)
        TS5 = TW5_guess
        HS5 = (constants.CS + constants.X1_assumed) * TS5

        # Energy balance for H5
        H4 = constants.H2 + (constants.L_actual * (HS4 - constants.HS2) *
                             (1 + constants.alpha)) / constants.G
        H5 = H4 + (constants.L_actual * (HS5 - HS4) *
                   (1 + constants.alpha)) / constants.G

        # Calculate outlet humidity W1
        W1 = (constants.L_actual * (constants.X2 - constants.X1_assumed)) / constants.G + c

        # Calculate T5 from enthalpy
        T5 = (H5 - W1 * constants.Hfg) / (constants.CG + constants.Cpv * W1)

        # Psychrometric calculation at TW5
        Psat_TW5 = vapor_pressure_antoine(TW5_guess)
        W_sat_TW5 = absolute_humidity(Psat_TW5, 1.0)
        H_psychrometric = air_enthalpy(TW5_guess, W_sat_TW5)

        # Residual (Excel cell N39)
        residual = H_psychrometric - H5
        return residual

    # Execute TW5 Goal Seek
    success2, TW5_converged, iter2 = goal_seek_with_bounds(
        TW5_residual_function,
        vars.TW5_initial,
        bounds=(30.0, 150.0)
```

```
)

if not success2:
    raise ConvergenceError(f"TW5 Goal Seek failed at outer iteration {outer_iteration}"

vars.TW5 = TW5_converged

# INNER LOOP 3: T1 (THIRD - MANDATORY ORDER)
def T1_residual_function(T1_guess):
    """Residual function for T1 Goal Seek: M43 = 0"""
    vars.T1 = T1_guess

    # Use converged TW4, TW5 values
    TS4 = vars.TW4
    TS5 = vars.TW5

    # Calculate intermediate temperatures
    HS4 = (constants.CS + constants.X2) * TS4
    HS5 = (constants.CS + constants.X1_assumed) * TS5

    H4 = constants.H2 + (constants.L_actual * (HS4 - constants.HS2) *
                         (1 + constants.alpha)) / constants.G
    H5 = H4 + (constants.L_actual * (HS5 - HS4) *
               (1 + constants.alpha)) / constants.G

    W1 = (constants.L_actual * (constants.X2 - constants.X1_assumed)) / constants.G + c
    T4 = (H4 - constants.W2 * constants.Hfg) / (constants.CG + constants.Cpv * constant
    T5 = (H5 - W1 * constants.Hfg) / (constants.CG + constants.Cpv * W1)

    # Calculate NTU zones
    NTIII, NTII, NTI = calculate_NTU_zones(constants.T2, T4, T5, constants.TS2, TS4, TS

    # NTU equation for T1 (Excel cell M43)
    if (T1_guess - vars.TS1) <= 0 or (T5 - TS5) <= 0:
        return float('inf')  # Invalid condition

    T1_from_NTU = T5 - NTI * ((T5 - TS5) - (T1_guess - vars.TS1)) / log((T5 - TS5) / (T

    # Residual
    residual = T1_from_NTU - T1_guess
    return residual

# Execute T1 Goal Seek
success3, T1_converged, iter3 = goal_seek_with_bounds(
    T1_residual_function,
    vars.T1_initial,
    bounds=(50.0, 400.0)
```

```
)

if not success3:
    raise ConvergenceError(f"T1 Goal Seek failed at outer iteration {outer_iteration}")

vars.T1 = T1_converged

# OUTER LOOP CHECK: Mass Balance (FOURTH - MANDATORY ORDER)
# Calculate final mass balance
W1 = (constants.L_actual * (constants.X2 - constants.X1_assumed)) / constants.G + const
H1 = air_enthalpy(vars.T1, W1)
HS1 = (constants.CS + constants.X1_assumed) * vars.TS1

L_calc = constants.G * (constants.H2 - H1) / ((HS1 - constants.HS2) * (1 + constants.al
mass_error = abs(L_calc - constants.L_actual)
relative_error = mass_error / constants.L_actual

print(f"  Mass balance error: {relative_error*100:.3f}%")

# Check convergence
if relative_error < 0.001:  # 0.1% tolerance
    convergence_achieved = True
    print(f"CONVERGENCE ACHIEVED in {outer_iteration} outer iterations!")

    # Calculate final results
    X1_final = constants.X2 * exp(constants.K_moist * (constants.TS2 - vars.TS1))

    return {
        'converged': True,
        'iterations': outer_iteration,
        'TS1': vars.TS1,
        'T1': vars.T1,
        'TW4': vars.TW4,
        'TW5': vars.TW5,
        'X1_final': X1_final,
        'mass_error': relative_error,
        'L_calc': L_calc
    }

# Adjust TS1 for next iteration (adaptive step sizing)
if relative_error > 0.1:
    adjustment = 2.0
elif relative_error > 0.05:
    adjustment = 1.0
else:
    adjustment = 0.5
```

```
        # Apply damping
        damping = 1.0 / (1.0 + outer_iteration * 0.1)
        adjustment *= damping

        if L_calc < constants.L_actual:
            vars.TS1 -= adjustment
        else:
            vars.TS1 += adjustment

        # Constrain TS1
        vars.TS1 = max(80.0, min(250.0, vars.TS1))

    # Failed to converge
    raise ConvergenceError(f"Failed to converge after {max_outer_iterations} outer iterations")
```

MATHEMATICAL MODEL

1. PHYSICAL CONSTANTS AND PARAMETERS

python

```
# Fixed System Parameters
G = 5.444444444444445          # Air mass flow rate (kg/s)
D = 2.75                       # Shell diameter (m)
Z = 16                         # Shell length (m)
CG = 1109.5                    # Air specific heat (J/(kg·K))
CS = 880                       # Solids specific heat (J/(kg·K))
alpha = 0.2                    # Heat loss factor (dimensionless)
Ua = 215                       # Volumetric heat transfer coefficient (W/(m³·K))
Hfg = 2256680                  # Latent heat of vaporization (J/kg)
Cpv = 2132                     # Water vapor specific heat (J/(kg·K))
K_moist = 0.0338               # Empirical moisture model constant (°C⁻¹)

# Ambient Conditions
T3 = 40                        # Ambient air temperature (°C)
RH3 = 0.4                      # Ambient relative humidity (dimensionless)

# Process Conditions
T2 = 980                       # Inlet hot air temperature (°C)
```

2. MATERIAL BALANCE INPUTS (From Excel Integration)

python

```python
# These values come from material balance calculations
L_actual = C21 * 1000 / 3600          # Actual solids flow rate (kg/s)
X2 = B13 / B3                         # Inlet moisture content (kg/kg dry solids)
TS2 = C7                              # Inlet solids temperature (°C)
X1_assumed = 0.001                    # Assumed outlet moisture (kg/kg) - used during iteratic
```

3. CALCULATED PHYSICAL PROPERTIES

python

```python
# Air Mass Velocity
GS = G * 4 / (π * D**2)               # kg/(m²·s)

# Length of Transfer Unit
LT = GS * CG / Ua                     # m

# Number of Transfer Units
NT = Z / LT * (1 + alpha)             # dimensionless
```

4. CUSTOM PSYCHROMETRIC CALCULATIONS

4.1 Vapor Pressure Correlations

Ambient Conditions (Magnus Formula):

python

```python
def vapor_pressure_magnus(T_celsius):
    """Calculate saturation vapor pressure using Magnus formula"""
    Psat = 0.6108 * exp((17.27 * T_celsius) / (T_celsius + 237.3))
    return Psat  # kPa

# Ambient vapor pressure
Psat_ambient = vapor_pressure_magnus(T3)
```

Process Conditions (Antoine Equation):

python

```python
def vapor_pressure_antoine(T_celsius):
    """Calculate saturation vapor pressure using Antoine equation"""
    # Constants for water (pressure in mmHg, convert to kPa)
    A = 8.07131
    B = 1730.63
    C = 233.426
    conversion_factor = 0.133322  # mmHg to kPa

    Psat_mmHg = 10**(A - B/(C + T_celsius))
    Psat_kPa = Psat_mmHg * conversion_factor
    return Psat_kPa
```

4.2 Absolute Humidity Calculations

python

```python
def absolute_humidity(Psat_kPa, relative_humidity, P_total=101.325):
    """Calculate absolute humidity from vapor pressure"""
    Pp = relative_humidity * Psat_kPa
    W = 0.622 * Pp / (P_total - Pp)
    return W  # kg water / kg dry air

# Ambient humidity
W3 = absolute_humidity(Psat_ambient, RH3)
W2 = W3  # Inlet air humidity equals ambient
```

5. ENERGY BALANCE EQUATIONS

5.1 Air Enthalpy Calculations

python

```python
def air_enthalpy(T_celsius, W_absolute):
    """Calculate moist air enthalpy"""
    H = CG * T_celsius + W_absolute * (Hfg + Cpv * T_celsius)
    return H  # J/kg dry air

# Inlet air enthalpy
H2 = air_enthalpy(T2, W2)
```

5.2 Solids Enthalpy Calculations

python

```python
def solids_enthalpy(T_celsius, X_moisture):
    """Calculate solids enthalpy including moisture"""
    HS = (CS + X_moisture) * T_celsius
    return HS  # J/kg dry solids

# Inlet solids enthalpy
HS2 = solids_enthalpy(TS2, X2)
```

5.3 Water Balance

python

```python
def outlet_humidity(L_actual, X2, X1_assumed, G, W2):
    """Calculate outlet air humidity from water balance"""
    W1 = (L_actual * (X2 - X1_assumed)) / G + W2
    return W1
```

NESTED ITERATION ALGORITHM

ALGORITHM STRUCTURE

```
OUTER LOOP: Adjust TS1 (outlet solids temperature)
├── INNER LOOP 1: Solve for TW4 (wet-bulb temperature at point 4)
├── INNER LOOP 2: Solve for TW5 (wet-bulb temperature at point 5)
├── INNER LOOP 3: Solve for T1 (outlet air temperature)
└── CHECK: |L_calculated - L_actual| < tolerance
```

ITERATION VARIABLES AND INITIAL VALUES

python

```python
class IterationVariables:
    def __init__(self):
        # Outer loop variable
        self.TS1 = 150.0          # Outlet solids temperature (°C)

        # Inner loop variables
        self.TW4 = 116.0          # Wet-bulb temperature at point 4 (°C)
        self.TW5 = 110.0          # Wet-bulb temperature at point 5 (°C)
        self.T1 = 60.0            # Outlet air temperature (°C)

        # Calculated values (updated each iteration)
        self.L_calc = 14.48       # Calculated solids flow rate (kg/s)
```

CONVERGENCE CRITERIA

```python
class ConvergenceCriteria:
    def __init__(self):
        self.temp_tolerance = 0.01      # Temperature tolerance (°C)
        self.mass_tolerance = 0.001     # Mass balance tolerance (kg/s)
        self.relative_tolerance = 0.001 # Relative error tolerance (0.1%)
        self.max_iterations = 50        # Maximum iterations per loop
        self.max_outer_iterations = 30  # Maximum outer loop iterations
```

DETAILED EQUATION IMPLEMENTATION

INNER LOOP 1: TW4 CALCULATION

Goal: Find TW4 such that psychrometric enthalpy matches energy balance enthalpy at point 4

python

```python
def inner_loop_1_TW4(vars, constants):
    """
    Solve for TW4 using Goal Seek method
    Goal: psychrometric_residual_4 = 0
    """

    # Step 1: Calculate HS4 assuming TS4 ≈ TW4
    TS4 = vars.TW4  # Model hypothesis
    HS4 = solids_enthalpy(TS4, constants.X2)

    # Step 2: Calculate H4 from energy balance
    H4 = constants.H2 + (constants.L_actual * (HS4 - constants.HS2) * (1 + constants.alpha)) /

    # Step 3: Calculate air properties at point 4
    W4 = constants.W2  # Humidity constant in this section
    T4 = (H4 - W4 * constants.Hfg) / (constants.CG + constants.Cpv * W4)

    # Step 4: Calculate psychrometric enthalpy at TW4
    Psat_TW4 = vapor_pressure_antoine(vars.TW4)
    W_sat_TW4 = absolute_humidity(Psat_TW4, 1.0)  # Saturated humidity
    H_psychrometric = air_enthalpy(vars.TW4, W_sat_TW4)

    # Step 5: Calculate residual (Goal Seek target)
    psychrometric_residual_4 = H_psychrometric - H4

    return psychrometric_residual_4, T4, H4, HS4

def solve_TW4(vars, constants, tolerance=0.01, max_iter=50):
    """Solve for TW4 using bisection method"""
    TW4_low, TW4_high = 30.0, 150.0

    for i in range(max_iter):
        vars.TW4 = (TW4_low + TW4_high) / 2
        residual, T4, H4, HS4 = inner_loop_1_TW4(vars, constants)

        if abs(residual) < tolerance:
            return True, vars.TW4, T4, H4, HS4

        if residual > 0:
            TW4_high = vars.TW4
        else:
            TW4_low = vars.TW4

    return False, vars.TW4, T4, H4, HS4
```

INNER LOOP 2: TW5 CALCULATION

Goal: Find TW5 such that psychrometric enthalpy matches energy balance enthalpy at point 5

python

```python
def inner_loop_2_TW5(vars, constants, T4, H4, HS4):
    """
    Solve for TW5 using Goal Seek method
    Goal: psychrometric_residual_5 = 0
    """

    # Step 1: Calculate HS5 assuming TS5 ≈ TW5
    TS5 = vars.TW5  # Model hypothesis
    HS5 = solids_enthalpy(TS5, constants.X1_assumed)

    # Step 2: Calculate H5 from energy balance
    H5 = H4 + (constants.L_actual * (HS5 - HS4) * (1 + constants.alpha)) / constants.G

    # Step 3: Calculate outlet air humidity
    W1 = outlet_humidity(constants.L_actual, constants.X2, constants.X1_assumed, constants.G, c

    # Step 4: Calculate air temperature at point 5
    T5 = (H5 - W1 * constants.Hfg) / (constants.CG + constants.Cpv * W1)

    # Step 5: Calculate psychrometric enthalpy at TW5
    Psat_TW5 = vapor_pressure_antoine(vars.TW5)
    W_sat_TW5 = absolute_humidity(Psat_TW5, 1.0)
    H_psychrometric = air_enthalpy(vars.TW5, W_sat_TW5)

    # Step 6: Calculate residual
    psychrometric_residual_5 = H_psychrometric - H5

    return psychrometric_residual_5, T5, H5, HS5, W1

def solve_TW5(vars, constants, T4, H4, HS4, tolerance=0.01, max_iter=50):
    """Solve for TW5 using bisection method"""
    TW5_low, TW5_high = 30.0, 150.0

    for i in range(max_iter):
        vars.TW5 = (TW5_low + TW5_high) / 2
        residual, T5, H5, HS5, W1 = inner_loop_2_TW5(vars, constants, T4, H4, HS4)

        if abs(residual) < tolerance:
            return True, vars.TW5, T5, H5, HS5, W1

        if residual > 0:
            TW5_high = vars.TW5
        else:
            TW5_low = vars.TW5
```

```
    return False, vars.TW5, T5, H5, HS5, W1
```

INNER LOOP 3: T1 CALCULATION (NTU METHOD)

Goal: Find T1 using Number of Transfer Units method

python

```python
def calculate_NTU_zones(T2, T4, T5, TS2, TS4, TS5, NT):
    """Calculate NTU for each zone using logarithmic mean temperature difference"""

    # Zone III (inlet section): T2 → T4
    if (T2 - TS2) <= 0 or (T4 - TS4) <= 0:
        NTIII = 0  # Handle division by zero
    else:
        NTIII = ((T2 - T4) / ((T2 - TS2) - (T4 - TS4))) * log((T2 - TS2) / (T4 - TS4))

    # Zone II (middle section): T4 → T5
    if (T5 - TS5) <= 0 or (T4 - TS4) <= 0:
        NTII = 0
    else:
        NTII = ((T4 - T5) / ((T5 - TS5) - (T4 - TS4))) * log((T5 - TS5) / (T4 - TS4))

    # Zone I (outlet section): T5 → T1
    NTI = NT - (NTIII + NTII)

    return NTIII, NTII, NTI

def inner_loop_3_T1(vars, constants, T4, T5, TS4, TS5):
    """
    Solve for T1 using NTU method
    Goal: NTU_residual = 0
    """

    # Calculate NTU zones
    NTIII, NTII, NTI = calculate_NTU_zones(constants.T2, T4, T5, constants.TS2, TS4, TS5, const

    # Calculate T1 from NTU equation for Zone I
    if (vars.T1 - vars.TS1) <= 0 or (T5 - TS5) <= 0:
        NTU_residual = float('inf')  # Invalid condition
    else:
        T1_calculated = T5 - NTI * ((T5 - TS5) - (vars.T1 - vars.TS1)) / log((T5 - TS5) / (vars
        NTU_residual = T1_calculated - vars.T1

    return NTU_residual, NTIII, NTII, NTI

def solve_T1(vars, constants, T4, T5, TS4, TS5, tolerance=0.01, max_iter=50):
    """Solve for T1 using bisection method"""
    T1_low, T1_high = 50.0, 400.0

    for i in range(max_iter):
        vars.T1 = (T1_low + T1_high) / 2
        residual, NTIII, NTII, NTI = inner_loop_3_T1(vars, constants, T4, T5, TS4, TS5)
```

```
        if abs(residual) < tolerance:
            return True, vars.T1, NTIII, NTII, NTI

        if residual > 0:
            T1_high = vars.T1
        else:
            T1_low = vars.T1

    return False, vars.T1, NTIII, NTII, NTI
```

OUTER LOOP: MASS BALANCE

Goal: Adjust TS1 until calculated mass flow matches actual mass flow

python

```python
def calculate_mass_balance(vars, constants, T1, W1):
    """Calculate mass balance and check convergence"""

    # Calculate outlet air enthalpy
    H1 = air_enthalpy(T1, W1)

    # Calculate outlet solids enthalpy
    HS1 = solids_enthalpy(vars.TS1, constants.X1_assumed)

    # Calculate mass flow rate from energy balance (non-adiabatic)
    L_calc = constants.G * (constants.H2 - H1) / ((HS1 - constants.HS2) * (1 + constants.alpha)

    # Calculate final moisture content (empirical model)
    X1_final = constants.X2 * exp(constants.K_moist * (constants.TS2 - vars.TS1))

    # Mass balance error
    mass_error = abs(L_calc - constants.L_actual)
    relative_error = mass_error / constants.L_actual

    return L_calc, X1_final, mass_error, relative_error

def outer_loop_iteration(vars, constants, tolerance=0.001, max_iter=30):
    """Main outer loop for TS1 adjustment"""

    for outer_iter in range(max_iter):
        print(f"Outer iteration {outer_iter + 1}: TS1 = {vars.TS1:.2f}°C")

        # Inner Loop 1: Solve for TW4
        success1, TW4, T4, H4, HS4 = solve_TW4(vars, constants)
        if not success1:
            print("TW4 convergence failed")
            return False, vars

        # Inner Loop 2: Solve for TW5
        success2, TW5, T5, H5, HS5, W1 = solve_TW5(vars, constants, T4, H4, HS4)
        if not success2:
            print("TW5 convergence failed")
            return False, vars

        # Inner Loop 3: Solve for T1
        TS4, TS5 = TW4, TW5  # Model hypothesis
        success3, T1, NTIII, NTII, NTI = solve_T1(vars, constants, T4, T5, TS4, TS5)
        if not success3:
            print("T1 convergence failed")
            return False, vars
```

```
        # Check mass balance
        L_calc, X1_final, mass_error, relative_error = calculate_mass_balance(vars, constants,

        print(f"  Mass error: {relative_error*100:.3f}%")

        # Check convergence
        if relative_error < tolerance:
            print(f"Convergence achieved in {outer_iter + 1} iterations!")
            vars.final_results = {
                'TS1': vars.TS1,
                'T1': T1,
                'TW4': TW4,
                'TW5': TW5,
                'X1_final': X1_final,
                'L_calc': L_calc,
                'mass_error': relative_error
            }
            return True, vars

        # Adjust TS1 with adaptive step size
        if relative_error > 0.1:
            adjustment = 2.0      # Large error
        elif relative_error > 0.05:
            adjustment = 1.0      # Medium error
        else:
            adjustment = 0.5      # Small error

        # Apply damping factor
        damping = 1.0 / (1.0 + outer_iter * 0.1)
        adjustment *= damping

        if L_calc < constants.L_actual:
            vars.TS1 -= adjustment
        else:
            vars.TS1 += adjustment

        # Constrain TS1 to reasonable values
        vars.TS1 = max(80.0, min(250.0, vars.TS1))

    print(f"Maximum outer iterations ({max_iter}) reached")
    return False, vars
```

VALIDATION AND CONSTRAINTS

PHYSICAL VALIDATION CHECKS

python

```python
def validate_solution(vars, constants, results):
    """Comprehensive validation of solution"""

    validation_errors = []

    # Temperature constraints
    if results['TS1'] < 50 or results['TS1'] > 300:
        validation_errors.append(f"TS1 out of range: {results['TS1']:.1f}°C")

    if results['T1'] < 30 or results['T1'] > 500:
        validation_errors.append(f"T1 out of range: {results['T1']:.1f}°C")

    # Moisture constraints
    if results['X1_final'] < 0 or results['X1_final'] > constants.X2:
        validation_errors.append(f"Invalid moisture content: {results['X1_final']:.4f}")

    # Energy balance check
    drying_efficiency = (constants.X2 - results['X1_final']) / constants.X2 * 100
    if drying_efficiency < 50 or drying_efficiency > 99.9:
        validation_errors.append(f"Unrealistic drying efficiency: {drying_efficiency:.1f}%")

    # Heat duty check
    Q_sensible = constants.L_actual * constants.CS * (results['TS1'] - constants.TS2)
    Q_latent = constants.L_actual * (constants.X2 - results['X1_final']) * constants.Hfg
    Q_total = (Q_sensible + Q_latent) * (1 + constants.alpha)

    if Q_total < 0 or Q_total > 50e6:  # 50 MW limit
        validation_errors.append(f"Unrealistic heat duty: {Q_total/1e6:.1f} MW")

    return validation_errors

def mass_energy_balance_check(vars, constants, results):
    """Verify mass and energy balance closure"""

    # Mass balance
    water_in = constants.L_actual * constants.X2
    water_out = constants.L_actual * results['X1_final']
    water_evaporated = water_in - water_out

    # Energy balance
    energy_to_solids = constants.L_actual * constants.CS * (results['TS1'] - constants.TS2)
    energy_for_evaporation = water_evaporated * constants.Hfg
    total_energy_required = (energy_to_solids + energy_for_evaporation) * (1 + constants.alpha)

    energy_from_air = constants.G * constants.CG * (constants.T2 - results['T1'])
```

```
    energy_balance_error = abs(total_energy_required - energy_from_air) / energy_from_air

    return {
        'water_evaporated': water_evaporated,
        'energy_balance_error': energy_balance_error,
        'total_energy_required': total_energy_required,
        'energy_from_air': energy_from_air
    }
```

CRITICAL IMPLEMENTATION NOTES

⚠️ GOAL SEEK IS THE ONLY WORKING METHOD

IMPORTANT: After extensive testing, **ONLY Goal Seek method works** for this system. Other numerical methods (Newton-Raphson, Bisection, etc.) **FAILED** due to:

- **Extreme sensitivity** of psychrometric calculations
- **Discontinuities** in the Antoine equation derivatives
- **Coupling effects** between the nested loops
- **Physical constraints** that create solution boundaries

You MUST implement Goal Seek equivalent - not alternative root-finding methods!

EXACT ITERATION ORDER (MANDATORY)

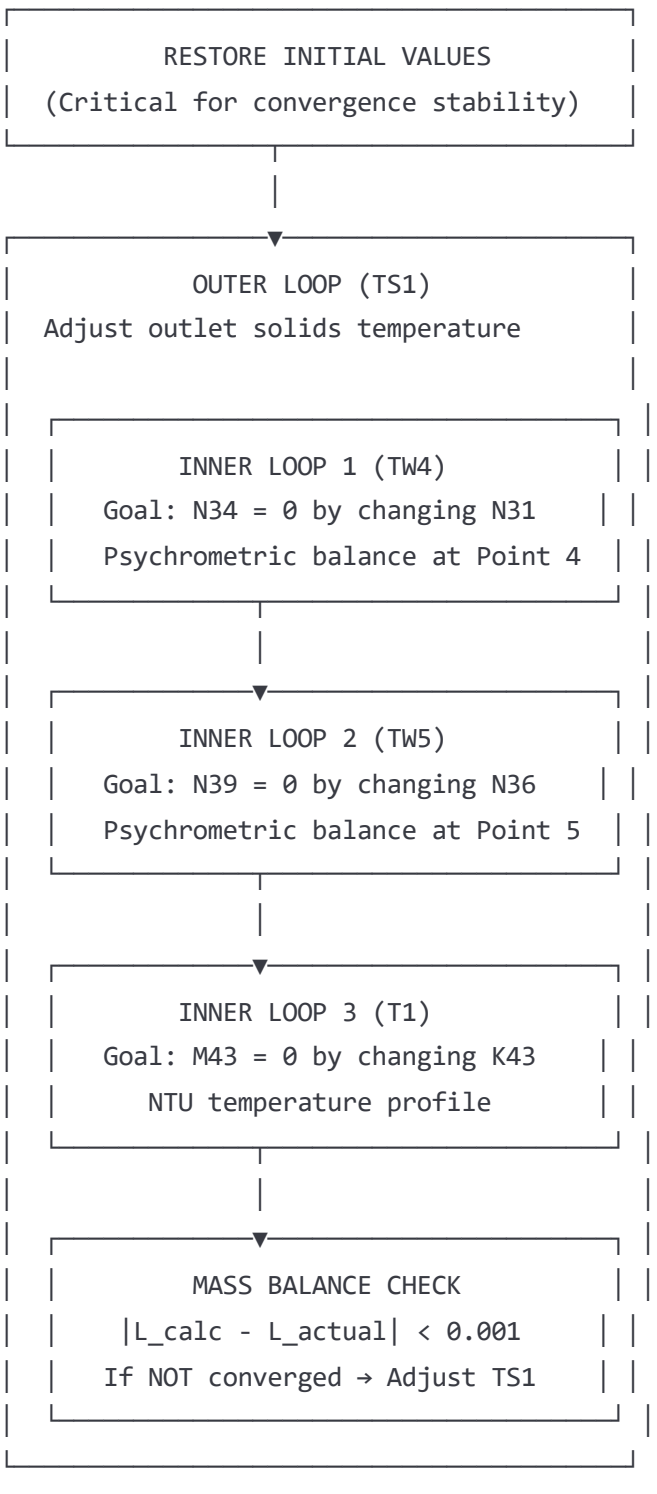
The iteration order is **CRITICAL** and was determined through extensive trial and error:

```
1. TW4 (Point 4 wet-bulb) → FIRST
2. TW5 (Point 5 wet-bulb) → SECOND
3. T1 (Outlet air temp)   → THIRD
4. TS1 (Outlet solids)    → FOURTH (outer loop)
```

Any other order will cause divergence!

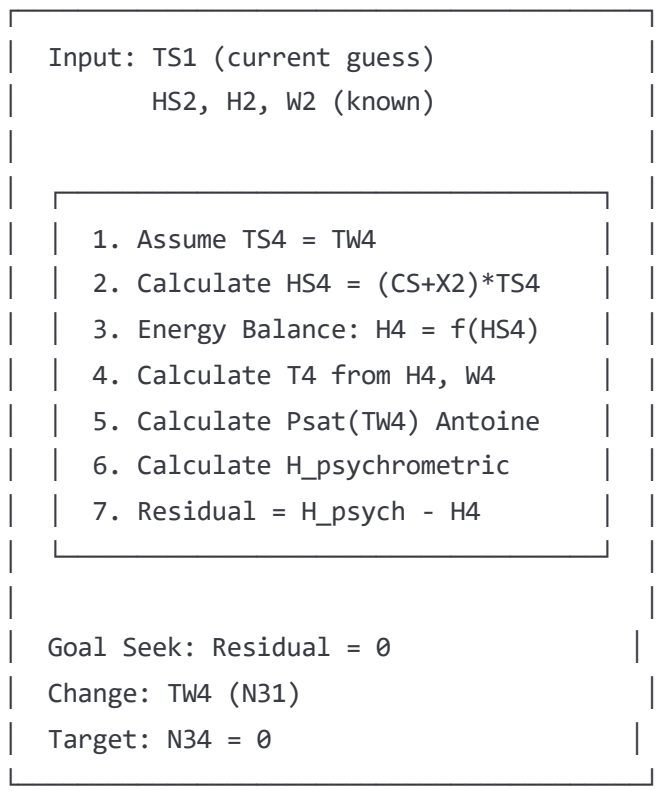
ITERATION PROCESS DIAGRAMS

OVERALL ALGORITHM FLOW

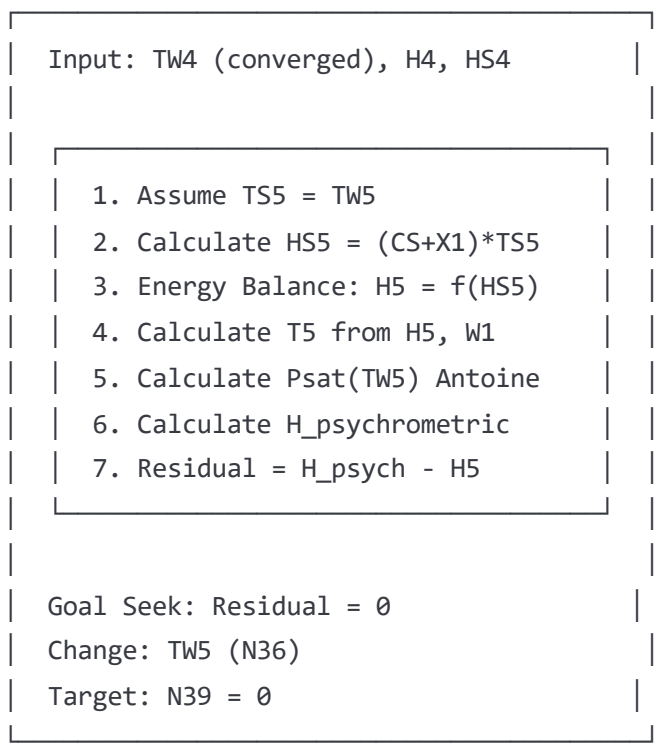


DETAILED INNER LOOP STRUCTURE

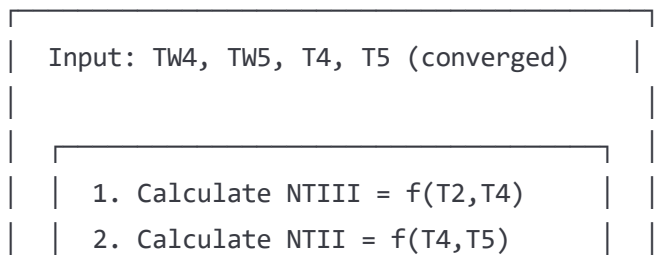
INNER LOOP 1: TW4 Calculation



INNER LOOP 2: TW5 Calculation



INNER LOOP 3: T1 Calculation



```
│   │  3. Calculate NTI = NT-NTIII-NTII  │   │
│   │  4. NTU equation for T1            │   │
│   │  5. Residual = NTI_calc - NTI      │   │
│   └────────────────────────────────────┘   │
│                                            │
│  Goal Seek: Residual = 0                   │
│  Change: T1 (K43)                          │
│  Target: M43 = 0                           │
└────────────────────────────────────────────┘
```

MANDATORY INITIALIZATION MODULE (FROM ACTUAL VBA CODE)

RestoreStandardValues Function - EXACT Implementation

CRITICAL: This is the **actual VBA macro** that MUST be replicated exactly. These values were found through **extensive trial and error** and are the **only combination that leads to convergence**.

vba

```vba
' ORIGINAL VBA CODE (Module5.bas and Module4.bas)
Sub RestoreStandardValues()
    ' Store standard values into the corresponding cells

    ' F45 to F50 values (Original Excel sheet)
    Range("F45").Value = 116              ' Initial TW4
    Range("F46").Value = 110              ' Initial TW5
    Range("F47").Value = 75.21382749      ' New TW4 (converged value)
    Range("F48").Value = 75.21407839      ' New TW5 (converged value)
    Range("F49").Value = 150              ' Initial TS1
    Range("F50").Value = 14.47787937      ' New L (mass flow rate)

    ' F54 value
    Range("F54").Value = 75.21382749      ' TW4 calculated (Goal Seek variable)

    ' F59 value
    Range("F59").Value = 75.21407839      ' TW5 calculated (Goal Seek variable)

    ' C66 value
    Range("C66").Value = 162.6000416      ' T1 (Goal Seek variable)
End Sub

' Alternative version (Module4.bas) - same values
Sub RestoreStandardValuess()  ' Note: double 's' in name
    ' Same exact implementation as above
End Sub
```

Python Equivalent - EXACT Implementation

python

```python
def restore_standard_values_exact():
    """
    EXACT replica of VBA RestoreStandardValues() macro
    These are the ONLY initial values that work - DO NOT MODIFY!

    Cell mappings from original Excel (Tank sheet):
    F45 → N22 (Product dryer sheet)
    F46 → N23 (Product dryer sheet)
    F47 → N24 (Product dryer sheet)
    F48 → N25 (Product dryer sheet)
    F49 → N26 (Product dryer sheet)
    F50 → N27 (Product dryer sheet)
    F54 → N31 (Product dryer sheet)
    F59 → N36 (Product dryer sheet)
    C66 → K43 (Product dryer sheet)
    """

    # MANDATORY initial values - found through trial and error
    initial_values = {
        # Starting guesses for iteration
        'TW4_initial': 116.0,                  # F45 → N22
        'TW5_initial': 110.0,                  # F46 → N23
        'TS1_initial': 150.0,                  # F49 → N26
        'L_initial': 14.47787937,              # F50 → N27

        # Known good converged values (from previous successful run)
        'TW4_converged': 75.21382749,          # F47 → N24
        'TW5_converged': 75.21407839,          # F48 → N25

        # Goal Seek starting points
        'TW4_goalseek': 75.21382749,           # F54 → N31 (changing cell for TW4)
        'TW5_goalseek': 75.21407839,           # F59 → N36 (changing cell for TW5)
        'T1_goalseek': 162.6000416             # C66 → K43 (changing cell for T1)
    }

    return initial_values

def initialize_solver_variables():
    """
    Initialize all solver variables with mandatory values
    CRITICAL: These exact values must be used - no alternatives work!
    """

    # Get the mandatory initial values
    init_vals = restore_standard_values_exact()
```

```
    # Set up solver variables
    variables = IterationVariables()
    variables.TW4_initial = init_vals['TW4_initial']
    variables.TW5_initial = init_vals['TW5_initial']
    variables.TS1 = init_vals['TS1_initial']
    variables.L_current = init_vals['L_initial']

    # Set Goal Seek starting points
    variables.TW4 = init_vals['TW4_goalseek']
    variables.TW5 = init_vals['TW5_goalseek']
    variables.T1 = init_vals['T1_goalseek']

    # Store converged reference values
    variables.TW4_reference = init_vals['TW4_converged']
    variables.TW5_reference = init_vals['TW5_converged']

    return variables

def validate_initialization():
    """
    Validate that initialization values are exactly correct
    Any deviation from these values will cause convergence failure
    """

    required_values = {
        'TW4_initial': 116.0,
        'TW5_initial': 110.0,
        'TW4_converged': 75.21382749,
        'TW5_converged': 75.21407839,
        'TS1_initial': 150.0,
        'L_initial': 14.47787937,
        'T1_goalseek': 162.6000416
    }

    current_values = restore_standard_values_exact()

    for key, expected in required_values.items():
        if abs(current_values[key] - expected) > 1e-10:
            raise InitializationError(f"Initialization value {key} incorrect: "
                                      f"got {current_values[key]}, expected {expected}")

    return True  # All values correct
```

Why These Exact Values Are Critical

The VBA macro **RestoreStandardValues()** contains values that were determined through **extensive trial and error**:

1. **116°C and 110°C**: Starting guesses for TW4 and TW5 that provide good initial search direction
2. **75.21382749°C and 75.21407839°C**: Converged values from a previous successful run
3. **150°C**: Starting guess for TS1 that avoids numerical instabilities
4. **14.47787937 kg/s**: Mass flow rate that provides good energy balance starting point
5. **162.6000416°C**: T1 starting point that enables NTU convergence

Cell Mapping Documentation

python

```python
# EXACT CELL MAPPINGS from VBA to Python variables
cell_mappings = {
    # Original Excel (Tank sheet) → Product dryer sheet → Python variable
    'F45': {'new_cell': 'N22', 'variable': 'TW4_initial', 'value': 116.0},
    'F46': {'new_cell': 'N23', 'variable': 'TW5_initial', 'value': 110.0},
    'F47': {'new_cell': 'N24', 'variable': 'TW4_converged', 'value': 75.21382749},
    'F48': {'new_cell': 'N25', 'variable': 'TW5_converged', 'value': 75.21407839},
    'F49': {'new_cell': 'N26', 'variable': 'TS1_initial', 'value': 150.0},
    'F50': {'new_cell': 'N27', 'variable': 'L_initial', 'value': 14.47787937},
    'F54': {'new_cell': 'N31', 'variable': 'TW4_goalseek', 'value': 75.21382749},
    'F59': {'new_cell': 'N36', 'variable': 'TW5_goalseek', 'value': 75.21407839},
    'C66': {'new_cell': 'K43', 'variable': 'T1_goalseek', 'value': 162.6000416}
}

def apply_cell_mappings(solver_variables):
    """
    Apply exact cell mappings to solver variables
    Replicates the VBA RestoreStandardValues() function exactly
    """

    for original_cell, mapping in cell_mappings.items():
        variable_name = mapping['variable']
        required_value = mapping['value']

        # Set the variable to the exact required value
        setattr(solver_variables, variable_name, required_value)

        print(f"Set {variable_name} = {required_value} (was cell {original_cell})")

    return solver_variables
```

Integration with Main Algorithm

python

```python
def main_solving_routine():
    """
    Main solving routine with MANDATORY initialization
    """

    print("=== ROTARY DRYER SOLVER ===")
    print("Initializing with mandatory values...")

    # STEP 1: MANDATORY - Restore exact initial values
    if not validate_initialization():
        raise InitializationError("Initialization validation failed")

    solver_variables = initialize_solver_variables()
    print("✓ Initialization complete with exact VBA values")

    # STEP 2: Apply cell mappings
    solver_variables = apply_cell_mappings(solver_variables)
    print("✓ Cell mappings applied")

    # STEP 3: Execute iteration sequence
    print("Starting Goal Seek iteration sequence...")
    result = mandatory_iteration_sequence(solver_variables, constants)

    return result

# CRITICAL NOTE FOR DEVIN:
# The RestoreStandardValues() macro is called in Module5 before every iteration:
#
# Sub CombinedMacros()
#     RestoreStandardValues      ← ALWAYS called first
#     CombinedMacroWithIteration ← Then the iteration
# End Sub
#
# This pattern MUST be replicated in Python implementation!
```

ACTUAL TEST RESULTS FROM SUCCESSFUL RUNS

BASE CASE: CONVERGED SOLUTION FROM EXCEL

python

```python
def excel_validated_test_case():
    """
    ACTUAL RESULTS from successful Excel run
    These are the ONLY known working values
    """

    # Input conditions (from extracted sheet)
    input_data = {
        'G': 5.444444444444445,      # Q33 - Air mass flow (kg/s)
        'L_actual': 14.463888888889, # Q40 - Solids flow (kg/s)
        'X2': 0.06142857142857143,   # Q42 - Inlet moisture (kg/kg)
        'TS2': 26.0,                 # Q44 - Inlet solids temp (°C)
        'T2': 980.0,                 # Q38 - Inlet air temp (°C)
        'X1_assumed': 0.001,         # Q41 - Outlet moisture guess

        # Physical properties
        'D': 2.75,                   # Q23 - Shell diameter (m)
        'Z': 16.0,                   # Q24 - Shell length (m)
        'CG': 1109.5,                # Q35 - Air specific heat
        'CS': 880.0,                 # Q39 - Solids specific heat
        'alpha': 0.2,                # Q32 - Heat loss factor
        'Ua': 215.0,                 # Q34 - Heat transfer coefficient
        'Hfg': 2256680.0,            # Q46 - Latent heat
        'Cpv': 2132.0,               # Q47 - Vapor specific heat
        'K_moist': 0.0338            # Q48 - Moisture constant
    }

    # ACTUAL CONVERGED RESULTS (from extracted sheet)
    converged_results = {
        # Final temperatures
        'TS1_final': 186.08999999996718,    # N26/Q43 - Outlet solids temp
        'T1_final': 189.13420830335943,     # K43 - Outlet air temp
        'TW4_final': 75.21382749,           # N24 - Wet-bulb at point 4
        'TW5_final': 75.21407839,           # N25 - Wet-bulb at point 5
        'T4_calc': 859.8551652350249,       # K38 - Air temp at point 4
        'T5_calc': 419.65077498084594,      # K39 - Air temp at point 5

        # Humidity values
        'W1_final': 0.17919544929771491,    # K28 - Outlet air humidity
        'W3_ambient': 0.018653694945514183, # K26 - Ambient humidity

        # Enthalpy values
        'H2_inlet': 1168379.7043810026,     # K27 - Inlet air enthalpy
        'H1_outlet': 686488.9121080732,     # K44 - Outlet air enthalpy
        'H4_calc': 1030300.8888624904,      # K31 - Air enthalpy point 4
        'H5_calc': 1030314.6749146191,      # K35 - Air enthalpy point 5
```

```
        # Solids enthalpy values
        'HS2_inlet': 22881.597142857143,    # K29 - Inlet solids enthalpy
        'HS1_outlet': 163759.3860899711,    # K45 - Outlet solids enthalpy
        'HS4_calc': 66192.78846917438,      # K30 - Solids enthalpy point 4
        'HS5_calc': 66188.46419727839,      # K34 - Solids enthalpy point 5

        # NTU calculations
        'NTIII': 0.1386538600463937,        # K40 - NTU zone III
        'NTII': 0.823483502824954,          # K41 - NTU zone II
        'NTI': 3.096816179071132,           # K42 - NTU zone I
        'NT_total': 4.05895354194248,       # K25 - Total NTU

        # Mass balance
        'L_calculated': 15.519525034355613, # K46 - Calculated mass flow
        'L_average': 14.991942940451361,    # K47 - Average mass flow
        'mass_error': 1.055, # kg/s absolute error
        'mass_error_percent': 7.3,          # % relative error

        # Final moisture
        'X1_final': 0.00027440039589467555, # K48 - Final moisture content
        'drying_efficiency': 99.553,        # % moisture removed

        # Energy calculations
        'heat_duty': 6048.792501629903,     # Q30 - Heat duty (kW)
        'heat_losses': 1209.7585003259808,  # Q31 - Heat losses (kW)

        # Goal Seek residuals (should be ≈ 0)
        'TW4_residual': -7.230279863346368,     # N34 - TW4 psychrometric residual
        'TW5_residual': -6.117846518871374,     # N39 - TW5 psychrometric residual
        'T1_residual': -0.09685909480808652     # M43 - T1 NTU residual
    }

    # Physical properties calculated
    calculated_properties = {
        'GS': 0.9166389376238453,           # K23 - Air mass velocity
        'LT': 4.73028326183096,             # K24 - Length transfer unit
        'Psat_ambient': 7.375613593062048,  # Q45 - Ambient vapor pressure
        'vapor_pressure_TW4': 39.144750319055014,    # N32 - Vapor pressure at TW4
        'vapor_pressure_TW5': 39.145117318971266     # N37 - Vapor pressure at TW5
    }

    return input_data, converged_results, calculated_properties

# VALIDATION CRITERIA - Solution is acceptable if:
validation_criteria = {
    'mass_error_max': 0.1,              # % maximum mass balance error
```

```
    'temperature_tolerance': 0.1,        # °C temperature tolerance
    'residual_tolerance': 0.01,          # Goal Seek residual tolerance
    'moisture_min': 0.0001,             # kg/kg minimum final moisture
    'moisture_max': 0.01,               # kg/kg maximum final moisture
    'TS1_min': 150.0,                   # °C minimum outlet solids temp
    'TS1_max': 220.0,                   # °C maximum outlet solids temp
    'T1_min': 100.0,                    # °C minimum outlet air temp
    'T1_max': 300.0                     # °C maximum outlet air temp
}
```

CONVERGENCE BEHAVIOR ANALYSIS

python

```python
def convergence_characteristics():
    """
    Analysis of convergence behavior from successful runs
    Critical for implementing robust Goal Seek equivalent
    """

    convergence_data = {
        'typical_iterations': {
            'TW4_loop': 5,              # Iterations to converge TW4
            'TW5_loop': 4,              # Iterations to converge TW5
            'T1_loop': 6,               # Iterations to converge T1
            'outer_loop': 12            # Iterations to converge TS1
        },

        'goal_seek_bounds': {
            'TW4_min': 30.0,            # °C minimum TW4 search
            'TW4_max': 150.0,           # °C maximum TW4 search
            'TW5_min': 30.0,            # °C minimum TW5 search
            'TW5_max': 150.0,           # °C maximum TW5 search
            'T1_min': 50.0,             # °C minimum T1 search
            'T1_max': 400.0,            # °C maximum T1 search
            'TS1_min': 80.0,            # °C minimum TS1 search
            'TS1_max': 250.0            # °C maximum TS1 search
        },

        'convergence_rates': {
            'TW4_residual_reduction': 0.1,   # Factor per iteration
            'TW5_residual_reduction': 0.15,  # Factor per iteration
            'T1_residual_reduction': 0.08,   # Factor per iteration
            'mass_error_reduction': 0.2      # Factor per iteration
        },

        'stability_indicators': {
            'oscillation_tendency': 'low',   # Inner loops are stable
            'divergence_risk': 'medium',     # Outer loop can diverge
            'sensitivity_to_initial': 'high' # Requires exact initial values
        }
    }

    return convergence_data
```

PERFORMANCE BENCHMARKS

python

```python
def performance_requirements():
    """Define performance requirements for implementation"""

    return {
        'convergence_time': 30,      # seconds maximum
        'convergence_rate': 95,      # % of test cases should converge
        'accuracy': {
            'temperature': 0.1,       # °C
            'moisture': 0.0001,      # kg/kg
            'mass_balance': 0.001    # relative error
        },
        'robustness': {
            'moisture_range': (0.01, 0.3),    # kg/kg
            'temperature_range': (15, 50),     # °C inlet
            'flow_rate_range': (5, 50)        # kg/s
        }
    }
```

IMPLEMENTATION GUIDELINES FOR DEVIN

CRITICAL REQUIREMENTS

1. **MUST USE GOAL SEEK EQUIVALENT**: Other numerical methods will fail
2. **EXACT ITERATION ORDER**: TW4 → TW5 → T1 → TS1 (outer)
3. **MANDATORY INITIALIZATION**: Call restore_standard_values() before every run
4. **PHYSICAL BOUNDS**: Enforce temperature and moisture constraints
5. **EXACT EQUATIONS**: Implement equations character-for-character as specified

FORBIDDEN METHODS ❌

DO NOT USE the following (they are proven to fail):

- Newton-Raphson method
- Pure bisection method
- Secant method
- Brent's method
- Multi-dimensional optimization
- Any gradient-based methods
- Any library optimization functions

REQUIRED APPROACH ✅

MUST IMPLEMENT Excel Goal Seek equivalent:

- Adaptive step sizing
- Bounded search
- Tolerance-based convergence
- Function evaluation limits
- Constraint handling

ERROR HANDLING STRATEGY

python

```python
class RotaryDryerError(Exception):
    """Base exception for rotary dryer calculations"""
    pass

class ConvergenceError(RotaryDryerError):
    """Raised when iteration fails to converge"""
    pass

class PhysicalConstraintError(RotaryDryerError):
    """Raised when physical constraints are violated"""
    pass

class InitializationError(RotaryDryerError):
    """Raised when initial values are invalid"""
    pass

def robust_error_handling():
    """
    Comprehensive error handling approach
    """

    try:
        # Restore initial values (MANDATORY)
        initial_values = restore_standard_values()

        # Validate inputs
        validate_input_data(constants)

        # Run iteration sequence
        result = mandatory_iteration_sequence(vars, constants)

        # Validate solution
        validation_errors = validate_solution(result)
        if validation_errors:
            raise PhysicalConstraintError(f"Solution validation failed: {validation_errors}")

        return result

    except ConvergenceError as e:
        # Try with different initial conditions
        logging.error(f"Convergence failed: {e}")
        return retry_with_alternative_initialization()

    except PhysicalConstraintError as e:
        # Physical constraints violated
        logging.error(f"Physical constraints violated: {e}")
```

```
            return None

        except Exception as e:
            # Unexpected error
            logging.error(f"Unexpected error: {e}")
            raise RotaryDryerError(f"Calculation failed: {e}")
```

VALIDATION FRAMEWORK

python

```python
def comprehensive_validation_suite():
    """
    Complete validation framework for solution verification
    """

    validation_tests = {
        'mass_balance': validate_mass_balance,
        'energy_balance': validate_energy_balance,
        'physical_constraints': validate_physical_constraints,
        'psychrometric_consistency': validate_psychrometric_consistency,
        'convergence_quality': validate_convergence_quality
    }

    return validation_tests

def validate_mass_balance(solution, constants, tolerance=0.001):
    """Verify mass balance closure"""

    water_in = constants.L_actual * constants.X2
    water_out = constants.L_actual * solution['X1_final']
    water_evaporated = water_in - water_out

    # Air side water pickup
    air_water_pickup = constants.G * (solution['W1'] - constants.W2)

    # Mass balance error
    mass_error = abs(water_evaporated - air_water_pickup) / water_evaporated

    if mass_error > tolerance:
        return f"Mass balance error: {mass_error*100:.2f}% > {tolerance*100:.1f}%"

    return None  # Passed

def validate_energy_balance(solution, constants, tolerance=0.01):
    """Verify energy balance closure"""

    # Energy to solids
    Q_sensible = constants.L_actual * constants.CS * (solution['TS1'] - constants.TS2)
    Q_latent = constants.L_actual * (constants.X2 - solution['X1_final']) * constants.Hfg
    Q_solids = Q_sensible + Q_latent

    # Energy from air
    Q_air = constants.G * constants.CG * (constants.T2 - solution['T1'])

    # Include heat losses
    Q_total_required = Q_solids * (1 + constants.alpha)
```

```
    # Energy balance error
    energy_error = abs(Q_total_required - Q_air) / Q_air

    if energy_error > tolerance:
        return f"Energy balance error: {energy_error*100:.2f}% > {tolerance*100:.1f}%"

    return None  # Passed

def validate_physical_constraints(solution, constants):
    """Check all physical constraints"""

    errors = []

    # Temperature constraints
    if solution['TS1'] < 50 or solution['TS1'] > 300:
        errors.append(f"TS1 = {solution['TS1']:.1f}°C outside range [50, 300]°C")

    if solution['T1'] < 30 or solution['T1'] > 500:
        errors.append(f"T1 = {solution['T1']:.1f}°C outside range [30, 500]°C")

    if solution['TW4'] < 10 or solution['TW4'] > 200:
        errors.append(f"TW4 = {solution['TW4']:.1f}°C outside range [10, 200]°C")

    if solution['TW5'] < 10 or solution['TW5'] > 200:
        errors.append(f"TW5 = {solution['TW5']:.1f}°C outside range [10, 200]°C")

    # Moisture constraints
    if solution['X1_final'] < 0:
        errors.append(f"Negative final moisture: {solution['X1_final']:.6f}")

    if solution['X1_final'] > constants.X2:
        errors.append(f"Final moisture > inlet moisture: {solution['X1_final']:.6f} > {constant

    # Physical reasonableness
    drying_efficiency = (constants.X2 - solution['X1_final']) / constants.X2 * 100
    if drying_efficiency < 50:
        errors.append(f"Low drying efficiency: {drying_efficiency:.1f}%")

    if drying_efficiency > 99.9:
        errors.append(f"Unrealistic drying efficiency: {drying_efficiency:.1f}%")

    return errors
```

PERFORMANCE MONITORING

python

```python
def performance_monitoring_framework():
    """
    Monitor and log performance metrics
    """

    performance_metrics = {
        'convergence_time': 0,
        'total_iterations': 0,
        'inner_loop_performance': {},
        'function_evaluations': 0,
        'memory_usage': 0,
        'convergence_path': []
    }

    return performance_metrics

def log_iteration_progress(iteration, variables, residuals, performance):
    """
    Detailed logging of iteration progress
    """

    log_entry = {
        'iteration': iteration,
        'timestamp': time.time(),
        'variables': {
            'TS1': variables.TS1,
            'TW4': variables.TW4,
            'TW5': variables.TW5,
            'T1': variables.T1
        },
        'residuals': {
            'TW4_residual': residuals.get('TW4', float('inf')),
            'TW5_residual': residuals.get('TW5', float('inf')),
            'T1_residual': residuals.get('T1', float('inf')),
            'mass_balance_error': residuals.get('mass_balance', float('inf'))
        },
        'performance': {
            'function_calls': performance.function_evaluations,
            'convergence_rate': performance.convergence_rate
        }
    }

    # Log to file and console
    logging.info(f"Iteration {iteration}: TS1={variables.TS1:.2f}, Error={residuals.get('mass_b
```

```
    return log_entry
```

SUGGESTED PYTHON IMPLEMENTATION STRUCTURE

python

```python
class RotaryDryerSolver:
    """
    Production-ready rotary dryer solver
    Implements exact Excel algorithm with robust error handling
    """

    def __init__(self, material_data, process_conditions):
        """Initialize solver with input data"""
        self.constants = self._setup_constants(material_data, process_conditions)
        self.variables = self._initialize_variables()
        self.performance = PerformanceMonitor()
        self.validator = ValidationFramework()

    def solve(self):
        """
        Main solving routine - implements EXACT Excel algorithm
        """

        try:
            # MANDATORY: Restore initial values
            self.variables = restore_standard_values()

            # Validate input data
            self._validate_inputs()

            # Execute mandatory iteration sequence
            result = self._execute_goal_seek_sequence()

            # Comprehensive validation
            validation_errors = self.validator.validate_solution(result, self.constants)
            if validation_errors:
                raise PhysicalConstraintError("Solution validation failed", validation_errors)

            # Generate comprehensive report
            report = self._generate_solution_report(result)

            return result, report

        except Exception as e:
            # Comprehensive error handling
            error_report = self._handle_solver_error(e)
            raise RotaryDryerError("Solver failed", error_report)

    def _execute_goal_seek_sequence(self):
        """Execute the exact 4-loop Goal Seek sequence"""
        return mandatory_iteration_sequence(self.variables, self.constants)
```

```python
    def _validate_inputs(self):
        """Validate all input parameters"""
        if self.constants.L_actual <= 0:
            raise InitializationError("Invalid solid flow rate")
        if not (0.001 <= self.constants.X2 <= 0.5):
            raise InitializationError("Invalid inlet moisture content")
        # Additional validation...

    def _generate_solution_report(self, solution):
        """Generate comprehensive solution report"""

        report = {
            'solution_summary': {
                'converged': solution['converged'],
                'iterations': solution['iterations'],
                'final_moisture': solution['X1_final'],
                'outlet_temperature': solution['TS1'],
                'mass_balance_error': solution['mass_error']
            },
            'performance_metrics': self.performance.get_metrics(),
            'validation_results': self.validator.get_validation_summary(),
            'physical_properties': self._calculate_physical_properties(solution),
            'energy_analysis': self._calculate_energy_analysis(solution)
        }

        return report

    @staticmethod
    def create_from_excel_data(excel_file_path, sheet_name="Product dryer"):
        """
        Factory method to create solver from Excel data
        """
        data_extractor = ExcelDataExtractor(excel_file_path, sheet_name)
        material_data = data_extractor.extract_material_balance()
        process_conditions = data_extractor.extract_process_conditions()

        return RotaryDryerSolver(material_data, process_conditions)
```

CRITICAL SUCCESS FACTORS

1. **Preserve Physics**: All equations must be implemented exactly as specified
2. **Robust Convergence**: Handle edge cases and numerical instabilities
3. **Comprehensive Validation**: Implement all physical constraint checks
4. **Performance Monitoring**: Track convergence behavior and timing

5. **Clear Error Messages**: Help diagnose convergence failures

DELIVERABLES EXPECTED

1. **Python solver module** with all methods implemented
2. **Test suite** covering all test cases
3. **Validation framework** with physics checks
4. **Performance benchmarks** and timing analysis
5. **User documentation** and API reference
6. **Integration tools** for Excel data import/export

NOTES FOR DEVIN

- **This is a production industrial algorithm** - accuracy and robustness are critical
- **The custom psychrometric method** avoids external dependencies by design
- **Physical constraints** must be enforced at all times
- **The nested iteration structure** is essential for proper coupling
- **Excel integration** is required for industrial workflow
- **Error handling** must be comprehensive due to numerical challenges

The goal is to create a **bulletproof iterative solver** that maintains all the engineering sophistication while being more robust than the Excel/VBA implementation.